EDG ClangIR back end

Executive summary: status at milestone M5.1

November 2026 · branch `cir` at c16c2798d2

At a glance

80.8%

C pass rate

90.3%

C++ pass rate

0

back-end crashes in 30,201 compiles

1.00×

clang's instructions on 9 kernels

6 of 15

real-world units compile

37 of 39

gate steps pass

The short version

- **C and standard C++ compile to x86-64 through ClangIR and LLVM.** Neither C nor gcc is in the code generation path.
- **Milestones R to M5.1 are done** (Oct 9 to Nov 6). M6 and M7 remain.
- **Failures are diagnosed, not miscompiled** (93%). Most are GNU or target extensions, which come later.
- **Biggest gaps:** three builtins that block libstdc++ and Boost headers, and no debug info.
- **Next:** review M5.1, answer four questions, plan M6.

Milestones

Milestone	Dates (2026)	What it delivered	Status
R, M0, M1	Oct 9	Design, the ECIL format, `BACK_END_TYPE`, `cpfe-cir`, skeletons	done
M2	Oct 10	A C subset compiles and runs	done
M3	Oct 10–12	Core C++: classes, templates, structors, linkage	done
M4	Oct 12–19	Virtual functions, RTTI, exceptions, member pointers; `-O` levels	done
M5, M5.1	Oct 19 – Nov 6	Compile speed, bounded memory, optimization attributes, mangling	done
M6	next	Remaining gaps, user and developer documentation	not planned yet
M7	later	Release configuration, Microsoft mode, Windows, other targets	later
Extensions	after M7	GNU extensions, then MSVC extensions	later

How it works

`front end → writer → ECIL stream → library → ClangIR → LLVM → x86-64`

The writer

`src/ecil_*`, about 25K lines.

Streams each function out as soon as the front end is done with it.

The library

`cir_src/`, about 36K lines of C++17.

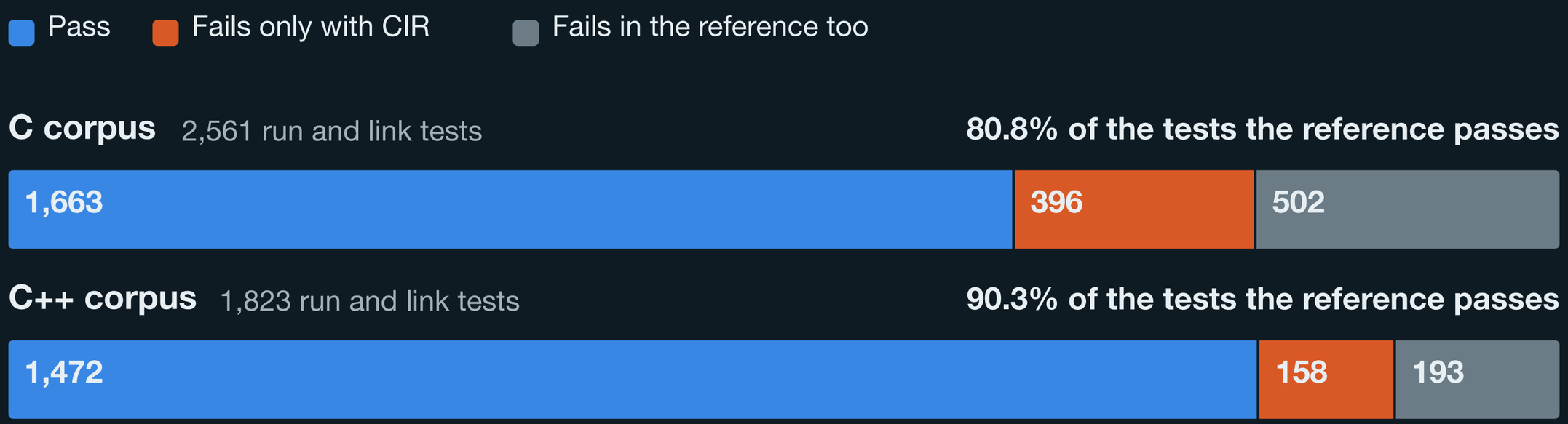
Builds ClangIR. Front-end changes do not rebuild it.

Scope today

x86-64 Linux, libstdc++.

In-process, or a second process that reads an `.ecil` file.

Test pass rates



Both corpora together: **85.0%** of the tests the reference passes (3,135 of 3,689).

Other test results

100%

of the smoke, regression and unit suites

70 of 70

link-and-run mixes with g++, and with clang

45,434

front-end regression tests per configuration: unchanged

Why tests still fail



Performance and code quality

1.00×

clang's instructions on 9 kernels; code size 1.01×

26–62%

of the pre-M5 library cost on real-world units

52%

peak memory with chunked lowering, 12,000-function unit

Biggest missing pieces

1. **Builtins that block real code.** `__builtin_constant_p`, `__atomic_*` and x86 add-with-carry block libstdc++ algorithms, containers, `<memory>` and `<regex>`, and four Boost libraries. Only 6 of 15 real-world units compile.
2. **Debug info.** None is emitted yet.
3. **GNU and target extensions.** Vector types, inline assembly, atomics and GNU attributes cause most failures. The brief puts them after standard C++.

Other gaps

— **Standard language:** coroutines, namespace-scope `thread_local` with dynamic initialization, C `_Complex`, VLAs, `_BitInt`, rare initializer forms.

— **Platforms:** x86-64 Linux only. AArch64 waits on upstream; Microsoft mode, Windows and macOS are M7.

— **Readiness:** no user or developer docs yet; a few attributes silently ignored.

— **Integration:** branch `cir` is local and not rebased on `main`.

Next steps

1. **Review M5.1** and answer the four open questions.
2. **Plan M6:** first the housekeeping, then the three builtins.
3. **Then in M6:** debug info, the standard-language gaps, code quality, clang-mode mangling, documentation.
4. **Plan the rebase** onto `main`.
5. **Then M7**, then the GNU and MSVC extensions.

Decisions needed

1. **Changelog:** move the older CIR entries to `src/pending_changes/`, and pick the GitHub issue that tags them.
2. **Memory ceilings:** tighten the C rows of the RSS check to 1.25 × the M5.1 values?
3. **clang4 clean-up:** delete the old trees there, or keep them?
4. **M6 order:** start with `__builtin_constant_p`, or with another item?

Risks

— **Upstream churn:** ClangIR changes fast; every pin bump needs re-checks.

— **Upstream scaling:** some passes are slow on extreme inputs.

— **Unverifiable facts:** a wrong "does not throw" fact silently skips cleanups.

— **Test infrastructure:** one shared x86-64 host; a full gate takes about 3.5 hours.